

BÁO CÁO BÀI TẬP LỚN: HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN vBay

HỆ THỐNG ĐẤU GIÁ TRỰC TUYẾN THỜI GIAN THỰC (vBay)

- **Môn học:** Lập trình nâng cao
- **Dự án:** vBay (Desktop App Client-Server Realtime)
- **Repository:** <https://github.com/cuongdz2k7/vbay>
- **Kiến trúc:** Multi-module Maven

1. MỤC TIÊU VÀ PHẠM VI THỰC HIỆN

1.1. Mục tiêu dự án

Dự án **vBay** xây dựng sàn đấu giá Desktop App theo mô hình Client-Server realtime. Bài toán chính không chỉ là cho người dùng đặt giá, mà còn phải bảo đảm nhiều người có thể thao tác cùng lúc mà dữ liệu giá, người thắng và số dư ví vẫn nhất quán. Vì vậy hệ thống tập trung vào **OOP/MVC/Design Patterns**, xử lý **concurrency**, truyền sự kiện qua **TCP Socket** và mô phỏng cơ chế ví ký quỹ an toàn.

Về nghiệp vụ, vBay không tách riêng tài khoản seller và bidder: một tài khoản **USER** có thể vừa đăng bán sản phẩm/tạo phiên đấu giá, vừa tham gia đặt giá ở các phiên khác. Người dùng có thể bid thủ công, đăng ký auto-bid, mua ngay bằng Buy Now và theo dõi diễn biến giá realtime. **ADMIN** giữ vai trò quản trị hệ thống như duyệt ví, khóa/cảnh cáo/kick user và xử lý các trường hợp vi phạm. Các tính năng Anti-sniping, reserve price, BidPriceChart và Self-Healing Scheduler giúp phiên đấu công bằng hơn, dễ quan sát hơn và ổn định hơn khi server gặp sự cố.

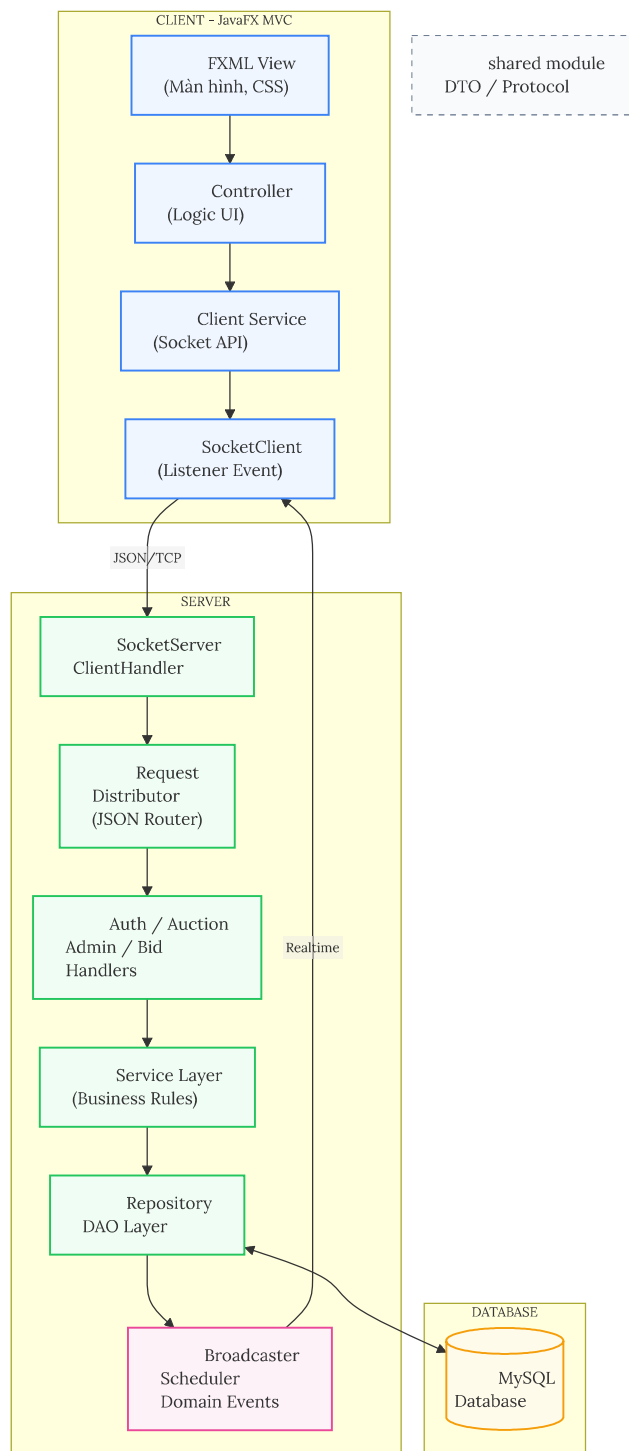
1.2. Phạm vi thực hiện

- **Môi trường:** JavaFX client, TCP server đa luồng, phân quyền **USER** và **ADMIN**; **USER** có thể đồng thời là người bán và người đấu giá.
- **Lưu trữ & test:** MySQL, H2 in-memory, JUnit 5/Mockito, GitHub Actions; không bao gồm cloud, thanh toán thật, SSL/TLS.
- **Luồng chính:** đăng ký/dăng nhập, tạo sản phẩm, tạo phiên đấu giá, đặt bid, auto-bid, mua ngay, theo dõi realtime, quản trị user và xử lý kết thúc phiên.

2. KIẾN TRÚC TỔNG THỂ CỦA HỆ THỐNG

2.1. Sơ đồ kiến trúc tổng thể

Hệ thống gồm 3 module Maven: **shared** (DTO/protocol), **client** (JavaFX MVC), **server** (service/repository/database). **shared** giữ hợp đồng giao tiếp để client và server không lệch model; **client** chỉ xử lý giao diện và gửi request; **server** chịu trách nhiệm xác thực, kiểm tra quyền, xử lý nghiệp vụ và ghi database. Cách chia này giúp phần đấu giá, ví tiền và realtime được kiểm soát tập trung ở server thay vì phân tán trong UI.



2.2. Mô tả chi tiết theo sơ đồ kiến trúc

- **Module & MVC:** `shared` chứa DTO/protocol dùng chung; `client` tách FXMLLoader, CSS và controller; `server` chia Distributor -> Service -> DAO để controller không chứa nghiệp vụ.
- **Realtime:** TCP port 3618 dùng Line-Delimited JSON để tránh dính gói tin; `SubscriptionRegistry` broadcast theo room `AUCTION_LIST`, `AUCTION_id`, `USER_id`.
- **Bidding pipeline:** `AuctionBidEngine` quyết định bid/winner/reserve, `BidResolutionApplier` ghi DB trong transaction có `FOR UPDATE`; scheduler quét lại DB khi khởi động hoặc theo chu kỳ để lập lịch lại các phiên còn hiệu lực sau crash.

3. CHỨC NĂNG ĐẠT ĐƯỢC THEO BAREM ĐIỂM

Nhóm chức năng	Hướng giải quyết	Lý do
Thiết kế OOP (2.5đ)	Entity đóng gói <code>private</code> ; ngoại lệ kế thừa <code>AppException</code> ; đa hình qua <code>AuctionChange</code> ; trừu tượng hóa bằng interface/service base.	Model nằm trong <code>shared</code> nên client-server dùng cùng hợp đồng dữ liệu, để mở rộng mà không phá code cũ.
Danh mục (2.5đ)	<code>Product</code> liên kết <code>categoryId</code> , danh mục cha-con lưu trong DB.	Danh mục có thể thêm/sửa ở runtime, tránh cây kế thừa Java cứng và khó bảo trì.
Design Pattern	Singleton (<code>ConnectionPool</code> , registries), Observer (socket rooms), Strategy (<code>AntiSnipePolicy</code>).	Singleton quản lý tài nguyên chung; Observer phù hợp realtime; Strategy cô lập luật gia hạn phút chót.
User/Product (3.0đ)	<code>AuthService</code> , Argon2, session socket; <code>USER</code> vừa có thể bán vừa có thể bid; admin ban/lock/kick/duyet ví.	Server kiểm soát quyền, dữ liệu đầu vào và nghiệp vụ nhạy cảm thay vì tin logic phía client.
Đấu giá (3.0đ)	<code>ManualBidService</code> , <code>AutobidService</code> , <code>BuyNowService</code> ; engine xử lý bid, reserve, winner, buy now.	Bidding có nhiều side effect tài chính nên tách engine tính toán khỏi lớp ghi database.
Lỗi & ngoại lệ (3.0đ)	<code>ValidationException</code> , <code>AuthenticationException</code> ; <code>RequestDistributor</code> trả <code>Respond(success=false)</code> ; transaction rollback.	Client nhận lỗi thống nhất, server không commit nửa vời khi bid/deposit/buy-now gặp lỗi.
Đồng thời (1.5đ)	<code>ClientHandler</code> riêng; bid/buy-now/auto-bid dùng transaction, <code>lockAuctionForUpdate</code> , <code>lockUserForUpdate</code> .	Khóa hàng auction/user giúp chống lost update, double spending và tranh chấp khi nhiều bidder cùng đặt giá.
Realtime (1.5đ)	Client subscribe room; server phát <code>RealtimeEvent</code> qua <code>RealtimeBroadcaster</code> .	Room-based broadcasting giảm tải mạng và hạn chế gửi nhầm dữ liệu riêng tư.
Client-server, MVC, DAO (2.0đ)	JavaFX/FXML controller -> <code>SocketClient</code> ; server JSON/TCP -> service -> repository JDBC.	Biên module rõ: UI chỉ điều phối hiển thị, service xử lý nghiệp vụ, DAO chịu trách nhiệm lưu trữ.
Maven, clean code (2.0đ)	Multi-module <code>client</code> / <code>server</code> / <code>shared</code> , Maven assembly fat JAR.	Build/phát hành nhất quán, dependency tách theo module và DTO giao tiếp không bị lặp.
Unit test & CI/CD (2.0đ)	JUnit 5/Mockito cho auth, auction, anti-snipe; GitHub Actions build/test/release.	Test tập trung vào logic rủi ro cao để giảm lỗi khi merge và đóng gói release.
Nâng cao (1.5đ)	Auto-bidding, anti-sniping, Buy Now, reserve price, <code>BidPriceChart</code> , escrow wallet, self-healing scheduler.	Các tính năng này làm phiên đấu công bằng hơn, trực quan hơn và an toàn hơn về trạng thái ví tiền.

4. KỸ THUẬT NÂNG CAO VÀ ĐIỂM NỔI BẬT

4.1. Tự thầu (Auto-bidding)

- Người dùng đặt `maxBidAmount`, hệ thống chỉ tăng giá công khai ở mức vừa đủ để giữ vị trí dẫn đầu. Hạn mức tối đa được giữ riêng tư và chỉ gửi qua response/event cá nhân.

4.2. Chống bắn tỉa (Anti-sniping)

- `AntiSnipePolicy` phát hiện bid hợp lệ sát giờ kết thúc và sinh `AntiSnipeAuctionExtensionChange` để gia hạn `ending_time`. Nhờ vậy bidder khác vẫn có thời gian phản hồi, tránh thắng thầu bằng cú đặt giá phút cuối.

4.3. Biểu đồ biến động giá (Bid history visualization)

- Client nhận `BID_HISTORY_ITEM_ADDED` và cập nhật trực tiếp `BidPriceChart`, không cần reload danh sách bid. Biểu đồ giúp quan sát tốc độ tăng giá và các thời điểm cạnh tranh mạnh trong phiên.

4.4. Buy Now và Reserve Price

- Buy Now cho phép chốt đơn ngay theo `buy_now_price`, kết thúc phiên và tạo payment `HELD`. `reserve_price` là giá sàn bảo vệ seller; nếu kết thúc chưa đạt reserve thì auction `FAILED` và hold được release.

4.5. Động cơ ký quỹ ví đóng băng (Wallet hold escrow engine)

- Ví tách `available_balance` và `hold_balance` để mô phỏng escrow. Khi winner thay đổi, hệ thống hold tiền winner mới và release người bị outbid trong cùng transaction, tránh một khoản tiền được dùng cho nhiều phiên.

4.6. Bộ lập lịch tự phục hồi (Self-Healing Scheduler)

- Scheduler chịu trách nhiệm mở/kết thúc phiên đấu giá đúng thời điểm. Khi server restart hoặc từng bị crash, hệ thống không phụ thuộc hoàn toàn vào lịch cũ trong bộ nhớ mà quét lại DB để tìm các auction còn hiệu lực, auction đã quá hạn nhưng chưa đóng và các phiên cần cập nhật trạng thái. Nhờ vậy lịch đấu giá được khôi phục từ dữ liệu bền vững, hạn chế tình trạng phiên bị bỏ quên hoặc kẹt trạng thái sau sự cố.